

GNN-Guided Graph Partitioning for Spatial CGRA Mapping

Capstone Project Final Report

Deniz Lopes Günes



Bachelor's in Informatics and Computing Engineering

U.Porto's Tutor: Professor João Cardoso
Company's Tutor: Professor Nuno Paulino

June 2026

Contents

Summary	2
Glossary	3
1 Introduction	5
1.1 Context	5
1.2 Technical concepts	5
1.3 Goals and expected results	7
1.4 Report structure	7
2 Methodology and main activities	7
2.1 Methodology	7
2.2 Stakeholders, roles and responsibilities	8
2.3 Activities carried out	8
2.3.1 First meeting (20 February 2026)	9
2.3.2 Second meeting (27 February 2026)	9
2.3.3 Third meeting (6 March 2026)	9
2.3.4 Fourth meeting (20 March 2026)	10
2.3.5 Fifth meeting (27 March 2026)	10
2.3.6 Sixth meeting (10 April 2026)	10
2.3.7 Seventh meeting (17 April 2026)	10
2.3.8 Eighth meeting (24 April 2026)	11
2.3.9 Ninth meeting (30 April 2026)	11
2.3.10 Tenth meeting (14 May 2026)	11
2.3.11 Eleventh and twelfth meetings: paper preparation	12
3 Development of the solution	12
3.1 Requirements and constraints	12
3.2 Architecture and technologies	13
3.2.1 Evolution of the technical approach	14
3.2.2 DFG representation and synthetic data generation	15
3.2.3 SAT-based mapper and ground-truth generation	15
3.2.4 GNN training and inference	16
3.2.5 Prediction repair and dependency representation	16
3.3 Developed solution	16
3.4 Validation	17
3.4.1 Validation strategy and experimental setup	17
3.4.2 Results and interpretation	17
4 Conclusions	18
4.1 Results achieved and individual contribution	18
4.2 Lessons learned	19
4.3 Limitations and future work	19
4.4 Acknowledgements	19

Summary

This report presents the work developed during the Capstone Project, focused on GNN-guided graph partitioning for spatial CGRA mapping. The project investigates how Graph Neural Networks can be used to guide the partitioning of Data Flow Graphs before mapping each partition onto a Coarse-Grained Reconfigurable Array using a SAT-based mapper. The proposed flow combines learned partition guidance with feasibility checks, aiming to reduce cut edges while ensuring that the resulting partitions remain mappable on the target CGRA.

Glossary

- AI** *Artificial Intelligence*; field of computing dedicated to techniques that enable systems with adaptive or learned behaviour.
- AIMACOV** National research project of the host team in which this work was integrated.
- ALAP** *As Late As Possible*; classical scheduling priority that schedules each operation at the latest cycle compatible with its data dependencies.
- ALU** *Arithmetic Logic Unit*; combinational unit inside a PE that performs the arithmetic and logical operations of the supported opcodes.
- ASAP** *As Soon As Possible*; classical scheduling priority that schedules each operation at the earliest cycle compatible with its data dependencies.
- CGRA** *Coarse-Grained Reconfigurable Array*; reconfigurable array composed of an array of configurable processing units.
- CGRA-ME** *CGRA Modelling and Exploration* [11]; open-source CGRA design framework considered as an architecture reference in this project.
- CTM** *Centre for Telecommunications and Multimedia*; INESC TEC centre where the project was hosted.
- DAG** *Directed Acyclic Graph*; directed graph with no cycles, used here as the underlying structure of a DFG.
- DFG** *Data Flow Graph*; directed graph that represents operations as nodes and data dependencies as edges.
- DQN** *Deep Q-Network*; reinforcement learning agent that approximates the Q-function with a deep neural network.
- FEUP** *Faculdade de Engenharia da Universidade do Porto*; Faculty of Engineering of the University of Porto.
- GNN** *Graph Neural Network*; neural network designed to process data structured as graphs.
- GraphSAGE** GNN architecture used as the classifier for partition prediction.
- INESC TEC** *Instituto de Engenharia de Sistemas e Computadores, Tecnologia e Ciência*; Institute for Systems and Computer Engineering, Technology and Science.
- JSON** *JavaScript Object Notation*; text-based format used to store and exchange structured data.

KMS *Kernel Mobility Schedule*; modulo-scheduling construct that bounds the cycles in which each operation may be issued, used by SAT-MapIt [13] to reduce the SAT search space.

L.EEC *Licenciatura em Engenharia Eletrotécnica e de Computadores*; Bachelor's Degree in Electrical and Computer Engineering at FEUP.

label0 Schedule-order label used in the earlier LISA-style priority experiments.

LISA *Graph Neural Network based Portable Mapping on Spatial Accelerators* [8]; GNN-based approach used to guide mapping on spatial accelerators.

LISA-GNN GNN module used in the prototype, inspired by the LISA [8] approach, to predict partition-related labels for DFG nodes.

LLVM Compiler infrastructure project providing modular and reusable compiler and toolchain technologies, used by the reference LISA flow.

mapper Component that searches for or verifies a mapping solution.

mapping Process of assigning the operations of a DFG to the physical resources of the CGRA.

NaviMap GNN plus deep-Q-network mapping approach for CGRAs, considered and ultimately discarded as a third leg of the prototype. [6]

one-shot Model in which a partition of the DFG is mapped onto a single spatial configuration of the CGRA.

PE *Processing Element*; individual processing unit within the CGRA.

SAT *Boolean Satisfiability*; problem of determining whether an assignment exists that satisfies a set of Boolean constraints.

SAT-MapIt SAT-based mapper for CGRAs [13], used as a technical reference in this project.

SAT-min-k Exhaustive ground-truth generation procedure that searches for the minimum number of partitions needed to map a DFG.

SMT *Satisfiability Modulo Theories*; generalisation of SAT with theories such as arithmetic, arrays and bit-vectors.

Z3 SMT/SAT solver [1] used to discharge the mapping constraints in the prototype.

1 Introduction

1.1 Context

This report documents the work carried out in the Capstone Project course unit, part of the Bachelor's in Informatics and Computing Engineering at the Faculty of Engineering of the University of Porto (FEUP), during the second semester of the 2025/2026 academic year.

The project was developed at **INESC TEC**, under the supervision of Professor Nuno Paulino and the tutorship of Professor João Cardoso. INESC TEC is an associate laboratory active in engineering and computer science. The work was integrated in the *Optical and Electronic Technologies* area of the CTM centre (Centre for Telecommunications and Multimedia), which hosts applied research in compilation and reconfigurable arrays, the areas in which this project is positioned.

The work is integrated in **AIMACOV**, an ongoing national research project of the host team. Within that effort I was integrated in a working group of five colleagues, accompanied by the project's grant holders, who provided technical guidance throughout the semester.

The goal was to investigate and implement an approach inspired by LISA [8], based on *Graph Neural Networks* (GNNs), for *mapping* of *Data Flow Graphs* (DFGs) onto *Coarse-Grained Reconfigurable Arrays* (CGRAs).

1.2 Technical concepts

Before going further, this subsection gives a quick overview of the main ideas behind the project, so the rest of the report can be read without the need of external references.

CGRA. A *Coarse-Grained Reconfigurable Array* is a hardware accelerator made of a grid of small *Processing Elements* (PEs), usually arranged as a 2D mesh. Each PE is basically an ALU with a tiny register file, and PEs talk to their direct neighbours through a fixed interconnect. The difference from an FPGA is the granularity: FPGAs are reconfigured at the bit level, while CGRAs are reconfigured at the word level. Each cycle, every PE is told which operation to run and where to read its operands from. This makes CGRAs a good fit for compute-heavy kernels with regular data flow, because they give up some of the flexibility of an FPGA in exchange for better energy efficiency and much smaller configuration data.

Mapping process. *Mapping* is the compilation step that decides which PE runs each operation and, if cycles are also considered, when. The input is normally a *Data Flow Graph* (DFG): a directed acyclic graph where nodes are operations and edges are data dependencies. The mapper has to respect three

kinds of constraints: *operation compatibility* (a node can only go on a PE that supports its opcode), *connectivity* (if $u \rightarrow v$ is a dependency, the PEs assigned to u and v must be directly connected, or the value has to be routed through other PEs or memory), and *resource limits* (no PE runs more than one operation in the same cycle, fan-in/fan-out budgets are respected, and so on). Mapping is NP-hard in general, so the literature has explored both exact approaches (SAT/ILP-based) and heuristics.

Graph Neural Networks. A *Graph Neural Network* (GNN) is a type of neural network that works directly on graphs. Each node starts with a feature vector and, through several rounds of message passing, mixes in information from its neighbours. After a few rounds, each node ends up with an embedding that captures both its own features and the structure around it, which can then be used to predict per-node, per-edge or per-graph properties. GNNs fit DFG-based problems naturally, because the input is already a graph and the things one usually wants to predict (scheduling priorities, distances, placement hints) live on the nodes.

LISA. LISA [8] is a GNN-based approach for mapping DFGs onto spatial accelerators. Instead of trying to solve the mapping itself, the GNN is trained on traces from a reference mapper to predict a few labels per DFG node, such as scheduling order and spatial/temporal distances to dependent nodes. These labels are then handed to a downstream mapper as constraints, which nudges it towards solutions similar to the reference flow while staying portable across different accelerator targets.

SAT-MapIt. SAT-MapIt [13] is an exact, SAT-based mapper for CGRAs. It encodes the mapping problem as Boolean constraints over placement variables, and uses a *Kernel Mobility Schedule* (KMS), a modulo-scheduling construct that bounds the cycles in which each operation can be issued, to shrink the search space before the SAT solver executes. The output is either a correct mapping (placement plus modulo schedule) or a proof that no mapping exists for the given resource budget.

In this project, LISA and SAT-MapIt are the two main references that the prototype combines: a LISA-style GNN provides per-node priorities, and a simplified, SAT-MapIt-inspired mapper checks each one-shot partition built from those priorities.

Related approaches. Mapping a DFG onto a CGRA is a well-known NP-hard problem, involving simultaneous placement and routing under resource constraints. Exact formulations such as Integer Linear Programming (ILP) and Boolean Satisfiability (SAT) guarantee optimal solutions but scale poorly, since as DFG and CGRA sizes grow, the search space expands exponentially. At the other extreme, heuristic mappers such as DRESC [9] use simulated annealing to solve placement, scheduling and routing simultaneously, trading optimality

guarantees for tractable compile time on larger graphs. SAT-based approaches such as Miyasaka et al. [10] encode the mapping constraints directly as a Boolean formula, while SAT-MapIt [13] narrows the search space by computing a Kernel Mobility Schedule and then applying SAT to find an optimal modulo schedule, but remain targeted at the single-configuration placement problem rather than multi-partition spatial mapping.

Recent work has demonstrated that GNNs can accelerate the downstream mapping process by predicting labels that reduce the search space for iterative mappers. GRAFT [7] also uses a GNN like LISA [8], but predicts a placement distribution through GNN-based subgraph matching to steer a progressive graph-based mapper, reporting a $64.5\times$ mapping speedup over LISA with comparable or better mapping quality on CGRAs up to 16×16 .

1.3 Goals and expected results

The goals defined for the project were the following:

- Study the state of the art on *mapping* of DFGs to CGRAs, in particular approaches that leverage AI to improve mapping times.
- Build a mapping flow that uses AI to produce high-quality mappings in a fraction of the time of a purely algorithmic approach.
- Evaluate the proposed flow and contrast it with other purely algorithmic strategies.

1.4 Report structure

The next section frames the methodology, stakeholders and activities. The development section describes the phases of the proposed flow, how each one works and how they link together. The experimental validation, known limitations and future work follow.

2 Methodology and main activities

2.1 Methodology

This project was conducted at INESC TEC in a hybrid model. I had the flexibility to work on the project remotely and to communicate with my supervisors and the rest of the team through a Slack[12] workspace that belongs to the team and to which I was invited.

Throughout the process, weekly meetings were held in at INESC TEC in which I presented what had been accomplished during the week and we discussed the results and what should be addressed during the following week. For these

meetings I prepared a short presentation of about 5-10 minutes. These meetings were an opportunity not only to clarify questions about the project and its implementation, but also to receive feedback from the supervisors regarding the current state of the prototype and the features to be integrated next. This is also why the project requirements evolved significantly over the semester: although the main objective was clear (use *AI* to guide the mapping process), the structure of the pipeline and exactly which state of the art approaches to follow were not fixed at the start, and the meetings were essential to decide which ideas were appropriate given the project’s goals and duration.

The development overall followed an iterative methodology[2], well suited to a research project of this nature. The project was developed inside a private GitLab[4] repository belonging to the team.

The technical resources used were:

- GitLab[4] for version control.
- Slack[12] for day-to-day communication with the team.

2.2 Stakeholders, roles and responsibilities

The project was developed individually by Deniz Lopes Günes.

- **Deniz Lopes Günes (author)**: literature review, implementation of the prototype and the benchmarks, execution of the end-to-end examples and the writing of the report.
- **Francisco Mouro Sampaio Nunes (team colleague, L.EEC)**: developed a project vertical to mine, responsible for generating the code used to configure the *CGR*A from the *mappings* produced.
- **Professor Nuno Paulino (main supervisor, INESC TEC)**: technical guidance, definition of goals, critical review of the work and validation of the results.
- **Guilherme Cruz (co-supervisor, INESC TEC)**: technical support and discussion of implementation strategies.
- **Henrique Teixeira (co-supervisor, INESC TEC)**: technical support and discussion of implementation strategies.
- **Professor João Cardoso (tutor, FEUP)**: academic accompaniment, alignment with the course unit’s requirements and critical review of the report.

2.3 Activities carried out

The technical activities can be grouped into the following blocks:

- **Study:** literature review on the state of the art on mapping DFGs to CGRAs, in particular approaches that leverage AI to improve mapping times.
- **Development:** development of the proposed GNN-Guided Graph Partitioning for Spatial CGRA Mapping framework.
- **Evaluation:** execution of a benchmark which compares the proposed approach with purely algorithmic methods, and evaluation of the results.
- **Scientific article:** preparation of a submission describing the mapping approach, which was submitted to the national conference INForum 2026 [5] (at the time of writing this report, under review)

The initial phases (study, prototyping with state-of-the-art tools) took place between the start of the semester and mid-March; the GNN-Guided Graph Partitioning for Spatial CGRA Mapping framework and benchmarks were consolidated in late May; the paper and report writing followed in the final weeks.

The remainder of this section walks through the project’s evolution meeting by meeting, which is the most faithful way to describe how the direction of the work was iteratively reshaped.

2.3.1 First meeting (20 February 2026)

The first meeting was used to re-introduce the team, agree on the communication channel (Slack) and align on the immediate reading material. I was tasked with surveying the state of the art on graph *mapping* for CGRAs, in particular GNN-based approaches, so that the next meeting could be grounded on concrete papers rather than on the generic proposal.

2.3.2 Second meeting (27 February 2026)

After the initial reading pass, the supervisors asked me to bring a dataset of DFGs of the kind used in the state-of-the-art papers, to review two or three of the most cited mappers, and to try out a conventional mapper as a baseline. The discussion also opened two questions that would stay with the project until the end: how to check that a *mapping* is valid, and how to represent a CGRA as a graph. The recommendation was to keep both representations as simple as possible.

2.3.3 Third meeting (6 March 2026)

By the third meeting I had cloned the LISA [8] DFG generator and produced a first synthetic dataset of graphs. Reproducing the LISA flow end-to-end ran into a practical blocker: LISA itself relies on *Gurobi*, another DFG Mapper, on top of an *LLVM* build, and the *LLVM* dependency could not be installed cleanly in the working environment. In parallel, I started reviewing SAT-MapIt [13], which uses a *Kernel Mobility Schedule* (KMS), a construct from *modulo scheduling*

that bounds the cycles in which each operation may be issued, to reduce the SAT search space. The key idea that came out of this meeting was the one that ended up defining the whole project: rather than picking one of the two approaches, combine them. Use the LISA-style GNN to add constraints to the graph, and let a SAT-based mapper consume those constraints.

2.3.4 Fourth meeting (20 March 2026)

Two things were settled in this meeting. First, the LISA flow was reproduced for the homogeneous case: 1000 DFGs were generated for a 4×4 CGRA and another 1000 for an 8×8 CGRA, although mapping all of them with the reference flow turned out to be impractical (roughly 40 graphs mapped in a full day of runtime for the 4×4 case). Second, I started reading *NaviMap* [6], a GNN-plus-*deep-Q-network* mapping approach, where a partial order-aware agent selects the next node to try to map and route, in an iterative fashion. We also discussed whether the LISA labels could be fed into a NaviMap-like agent. The idea was left open as a possible third leg of the prototype, to be evaluated in the following meeting after a deeper read of the paper.

2.3.5 Fifth meeting (27 March 2026)

This meeting was when the NaviMap direction was dropped. After a closer read, combining a *deep-Q-network* agent on top of the GNN labels turned out to add significant implementation and training complexity, having to handle different input shapes for the neural networks as well as a lack of open-source availability from NaviMap, without a clear advantage over the simpler LISA-plus-SAT-MapIt pairing for the scale of graphs we were targeting. The decision was therefore to set NaviMap aside and to concentrate the remaining effort on the LISA-style labels combined with the SAT mapper.

2.3.6 Sixth meeting (10 April 2026)

The discussion in this meeting was almost entirely about the *codegen* side of the project, led by Francisco. From my side, the main takeaway was that the mapping flow had to produce a clean, stable JSON output, so that the downstream code generation could consume it without further negotiation about the format.

2.3.7 Seventh meeting (17 April 2026)

At this point I had a Python CGRA class that is passed to the mapper, LISA was producing four labels per node (used as features), and the experimental SAT path was using two of those labels (temporal and spatial distance) on top of a *CGRA-ME* [11]-style template to produce reference mappings for that prototype stage. The meeting surfaced a real, recurring problem: the DFG formats used by LISA and by SAT-MapIt did not agree. LISA worked with a small set of operations (`const`, `load`, `store`, `add`, `mul`, `sub`, ...), whereas SAT-MapIt also

reasoned about branches, *live-ins/live-outs* and other entities. The action item was to consolidate a single input format and intermediate JSON, under the assumption that everything is one-shot, and to check whether SAT-MapIt could be coerced into a one-shot regime. This meeting was when we decided to simplify the problem by focusing on one-shot mapping with no temporal constraints.

2.3.8 Eighth meeting (24 April 2026)

This is the meeting in which the eventual shape of the prototype was decided. The problem on the table was that SAT-MapIt is built around *modulo scheduling*, its notion of an *initiation interval*, and the way it places operations across cycles all assume that regime, whereas LISA does not directly emit a SAT-MapIt KMS or a complete modulo schedule. It predicts labels such as schedule order and spatial/temporal distances that guide a downstream mapper. Plugging the LISA labels straight into SAT-MapIt was therefore not straightforward. Due to time constraints and the scope of the Capstone Project, the team decided to simplify and abstract the mapping into one-shot mapping. This would raise an issue, where the LISA labels would be pointless since all the nodes would be placed at timestep 0. My suggestion was to invert the approach: take a large DFG, run a learned model on it to obtain per-node features, use those features to *partition* the graph in such a way that cut dependencies are made explicit through synthetic memory operations, and then one-shot *map* each subgraph with the SAT mapper. The open question became how to perform the partition itself, whether by a heuristic or by another learned component.

2.3.9 Ninth meeting (30 April 2026)

The ninth meeting confirmed that the partitioned one-shot strategy worked. A greedy partitioner was already in place, not yet driven by a GNN, but by hand-crafted priorities such as *ASAP*, *ALAP* and edge count — whose objective was to minimise the number of edges cut at the partition boundary, and thus the amount of synthetic memory traffic needed to keep memory operations outside of the CGRA’s ALU array. The splitting was working and each subgraph was being mapped one-shot with the SAT-MapIt-inspired one-shot mapper to produce reference labels for this abstraction. From here the plan was clean: use the original large graph, augmented with the per-partition timesteps as an extra feature, to train a GNN that predicts the schedule order (*label0*); then feed *label0* into the partitioner to obtain better partitions, and finally validate each partition with the simplified SAT mapper.

2.3.10 Tenth meeting (14 May 2026)

At the tenth meeting, I presented the work done on the GNN-guided partitioner. A key detail was to generate better partitions for training data. An exhaustive algorithm was developed that iterates through all possible partitions in an upward sweep and, for each partition, checks if it is mappable with the

SAT mapper. If it is, the partition is stored as a candidate, and the algorithm continues to the next partition. The algorithm continues until all possible partitions have been checked, and at the end, the best partition (the one with the lowest amount of cut edges) is selected from the candidates. This exhaustive approach ensures that the best possible partition is found for each DFG, which can then be used to train the GNN-guided partitioner. Also, I brought up a detail that was discovered while running the first benchmark on the greedy GNN-guided partitioner. Although the GNN predictions would improve the partitioning, a large amount of DFGs were still unmappable. The reason was that the greedy partitioner was not aware of the CGRA’s topology and routing constraints, so it could produce partitions that were not embeddable onto the target hardware. The solution proposed in this meeting was to implement a new kind of topology-aware algorithmic partitioner that would be aware of the graph and CGRA topologies and would respect their routing constraints. This entailed a new step in the partitioning process, the repair step, where a partition would be checked with the SAT mapper and if it was not mappable, the partition would be repaired by moving nodes to other partitions until it was mappable. Combining this topology-aware partitioner with the GNN-guided predictions would allow it to produce partitions that were mappable and would reduce the number of cut edges, which was the main goal of the partitioning process.

2.3.11 Eleventh and twelfth meetings: paper preparation

The final meetings were dedicated to preparing and reviewing the scientific paper that resulted from the project, and are grouped here because they formed a continuous writing and revision process.

3 Development of the solution

3.1 Requirements and constraints

As a research project, the requirements were deliberately open-ended. The main goal was to investigate how AI could guide DFG-to-CGRA mapping and deliver a working prototype that could be evaluated within one semester.

Throughout the internship, requirements and constraints constantly evolved. The final prototype was shaped by the following needs and restrictions:

- **Research-team goal:** Use learned information to improve a mapping decision.
- **Integration need:** Accept and produce stable JSON so the parallel code-generation project could consume the resulting mappings.

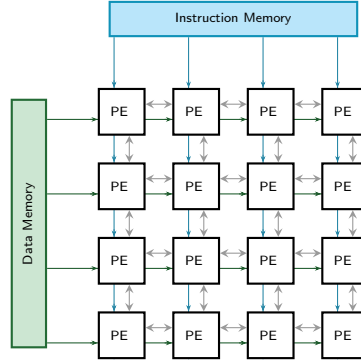


Figure 1: Targeted 4×4 homogeneous CGRA: 16 PEs with full ALU capability and memory access, with nearest-neighbour connections.

- **Functional need:** Accept a DFG and CGRA description, split graphs that do not fit in one configuration, validate every partition, and export PE assignments and operand connections.
- **Quality goal:** Reduce inter-partition cuts without making mapping time impractical, because every cut implies an additional value transfer.
- **Validation goal:** Compare the learned approach with exhaustive and heuristic alternatives using repeatable experiments.

Due to time constraints, the complexity of the problem, and the scope of the internship, some simplifications were assumed about how the CGRA and DFG were represented and processed, which do not reflect the full complexity of the real-world scenario. One-shot spatial mapping was assumed for each partition, internal dependencies required direct neighboring PEs, and cross-partition values were represented through synthetic memory operations. Our model assumes that the CGRA has a clock period long enough for every supported operation to complete in one clock cycle. Additionally, the DFG’s cut edges are represented as injected memory operations, which are not part of the original DFG. Finally, the DFG itself is assumed to be a directed acyclic graph, which is not always the case in real-world applications, representing a loop that is unrolled. The proposed flow does not do modulo scheduling nor does it determine a kernel; the whole DFG is assumed to be a single kernel.

The targeted CGRA for the final experiments was a homogeneous 4×4 array (Figure 1), and the DFGs were synthetic, generated from small kernel templates.

3.2 Architecture and technologies

The final framework is illustrated in Figure 2. The flow can be divided into two parts, the training phase and the inference phase. The training phase is

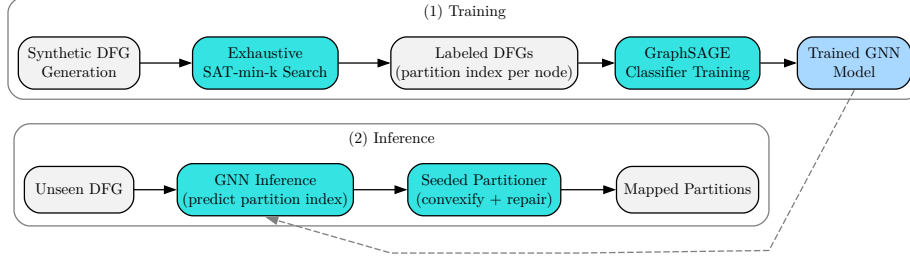


Figure 2: Overview of the proposed flow. The methodology is divided into two distinct phases: training the GNN and inference-time graph partitioning with SAT-based spatial validation.

responsible for generating synthetic DFGs, producing ground-truth labels with the exhaustive partitioner and mapper, and training the GNN model. The inference phase uses the trained GNN to predict partitions for new DFGs, which are then fed as seeds to the topology-aware mapper, which repairs infeasible predictions and validates the final partitions with the SAT mapper.

Technology-wise, the tool was developed in Python, which allowed fast experimentation and integration of graph, machine-learning and solver components, and has been the choice for this problem by the state of the art. The Z3 SMT solver was used to express and check placement constraints. GitLab was used for version control, and the actual graph/ML libraries used in the implementation were PyTorch Geometric for GNN training and inference, and NumPy for numerical operations.

3.2.1 Evolution of the technical approach

As has already been introduced by the sequence of meetings, the technical approach evolved through several iterations. After reaching the final idea for the flow (partitioning large graphs, training a model to predict partitions, then running a heuristic SAT-based mapper to validate the partitions), the next step was to evolve the technical approach. The first advancement was implementing the exhaustive SAT-min-k algorithm, which allowed us to produce better ground-truth labels to improve the GNN’s predictions. By implementing the pure greedy (list-scheduling) approach with the GNN-guided priorities, we concluded that the success rate was too low, and that the only successes were on simple graphs, which did not stress the routing constraints of the CGRA. Investigating the failures, we concluded that the GNN predictions were not enough to guarantee a feasible mapping, and that a repair step was needed. The repair step was implemented as a topology-aware partitioner that would split infeasible partitions at a feasible topological boundary until all partitions were feasible. For later evaluation we also implemented a pure raw GNN approach that would just take the GNN predictions and run the SAT-based mapper on them, without any repair step. Although this now becomes an iterative approach, with

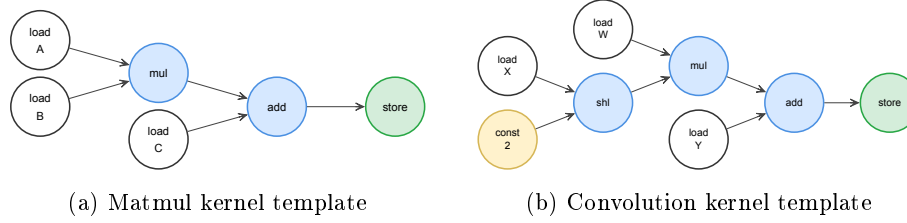


Figure 3: Kernel templates used as a base for DFG generation.

the GNN predictions the topology-aware partitioner is able to produce feasible partitions with low iteration counts, later presented in the results section.

3.2.2 DFG representation and synthetic data generation

To train neural networks, a large number of labelled examples is needed. Similar to what the LISA [8] approach does, we implemented a synthetic DFG generator capable of producing a large number of DFGs. Our generator, in contrast to the LISA generator which is purely stochastic, is based on a set of seed templates, which are then mutated, with tunable parameters, to produce a large number of DFGs. The two seed templates are based on the inner loop body of a matrix multiplication and convolution. The mutations are divided into three families: depth mutations, breadth mutations and neutral mutations. Depth mutations increase the length of dependency chains, breadth mutations increase the number of parallel operations and routing pressure, and neutral mutations diversify the operations without changing the topology.

3.2.3 SAT-based mapper and ground-truth generation

The SAT-based mapper that was implemented is a simplified version of the SAT-MapIt [13] mapper that only performs one-shot spatial mapping. The mapper receives a DFG and a CGRA description, and for each instruction, it chooses exactly one compatible PE. No PE may host more than one instruction, dependent instructions must be placed on connected PEs, and memory operations must use memory-capable PEs when the architecture is heterogeneous. If the Z3 solver can find a valid assignment, the mapper returns the mapping; otherwise, it returns an infeasible result.

Ground-truth generation for the GNN training was performed by the SAT-min-k algorithm, which is an exhaustive search for the minimum number of partitions needed to map a DFG. Starting from the minimum partition count based on CGRA capacity, the implementation enumerates ordered candidate assignments, rejects candidates that violate cheap structural checks, and SAT-validates the remaining partitions. After finding all feasible partitions, the one with the minimum partition count and edge cuts is selected as the ground-truth label. To control the quality of the output, parameters such as early stop on zero cut edges and maximum partition count are configurable.

3.2.4 GNN training and inference

The GNN model used in this work is a GraphSAGE [3] classifier that receives per-node structural and operation features, the same ones used in LISA [8], and predicts the partition index learned from SAT-min-k solutions. The model was trained on 1500 synthetic DFGs. The dataset was biased toward moderate graph sizes and low-to-medium parallelism to ensure that the SAT-min-k algorithm could produce ground-truth labels in a reasonable time. This choice means that the model may not generalize well to larger or more complex graphs.

During inference time, the GNN predicts partition indices for each node in an unseen DFG. The predictions are then passed as seeds to the topology-aware partitioner, which repairs infeasible predictions and produces valid partitions for SAT-based spatial validation.

3.2.5 Prediction repair and dependency representation

In an ideal scenario, the GNN is able to predict feasible partitions for any DFG. However, in practice, and with the limited time and computational resources available during training, the GNN was not able to learn a perfect mapping. As such, the predicted partitions may violate dependency order, exceed resource capacity, or fail spatial SAT validation. To address this, a topology-aware partitioner was implemented to repair the predicted partitions. The repair process first makes partition indices consistent with the DFG topology and compacts empty indices, enforcing that a consumer node cannot be placed before its producers. Then it tests each partition and splits infeasible ones at a feasible topological boundary until all remaining partitions can be mapped.

As mentioned before, cut edges are represented as synthetic store/load operations, which are injected into the DFG and are part of the mapping. This was the way the team decided to represent cross-partition dependencies, so that each partition becomes self-contained for one-shot mapping. The original edge is replaced by a store operation in the producer partition and a load operation in the consumer partition, which are connected to the same memory location. This way, the mapper can treat each partition independently, while still preserving the data flow between them.

3.3 Developed solution

The final solution is a complete pipeline that takes a DFG and a CGRA description, generates annotations that are fed into the GNN which predicts partitions, repairs infeasible predictions, and validates the final partitions with a SAT-based mapper, as described in Figure 2. The output is a set of valid partitions with PE assignments and operand connections, exported in JSON format for use in the code generation project.

Table 1: Structural statistics of the training (1500 graphs) and evaluation (150 graphs) dataset splits. All values reported as mean $\pm$ std (min–max).

Metric	Training	Evaluation
Nodes	22.51 ± 3.31 (16–28)	23.16 ± 2.92 (16–28)
Edges	21.77 ± 3.39 (15–29)	22.45 ± 2.97 (15–28)
Depth	8.19 ± 1.32 (6–13)	8.40 ± 1.30 (6–12)
Parallelism	4.26 ± 0.95 (2–8)	4.19 ± 0.97 (2–7)
Instr-Level Parallelism	2.21 ± 0.45 (1.3–4.0)	2.21 ± 0.48 (1.3–3.8)
Memory ratio	0.42 ± 0.07 (0.21–0.62)	0.42 ± 0.08 (0.25–0.60)

3.4 Validation

3.4.1 Validation strategy and experimental setup

To evaluate the proposed solution, a 4×4 homogeneous CGRA (Figure 1) was targeted. A total of 1650 DFGs were generated, with 1500 used for training and ground-truth generation, and 150 held out for the final evaluation. The graph characteristics are described in Table 1.

The evaluation compared five strategies: SAT-min-k, Greedy, Topological, GNN-raw, and GNN-guided. SAT-min-k is the exhaustive search for the minimum number of partitions, which serves as a ground-truth reference. Greedy is a simple list-scheduling baseline that maps nodes in topological order without any learned guidance. Topological is the repair-only strategy that uses the topology-aware partitioner to fix infeasible predictions. GNN-raw is the GNN prediction without any repair, and GNN-guided combines the GNN predictions with the repair step.

3.4.2 Results and interpretation

Tables 2a and 2b summarise the benchmark results. On the full 150-graph evaluation set, the greedy baseline mapped only 23 graphs (15.3%), while GNN-raw mapped 106 graphs (70.7%), the topology-only repair mapped 139 graphs (92.7%), and GNN-guided mapped 147 graphs (98.0%). The mean edge cuts for the full set were 3.22 for the unguided topological method, 2.15 for GNN-guided, and 1.31 for SAT-min-k. On the 150 graph set, the GNN-raw approach seemed to show a slight improvement over the topological method, with 1.74 mean cuts. To evaluate this improvement fairly, we restricted the comparison to the 106 graphs that GNN-raw could map in Table 2b. On this subset, GNN-guided achieved 1.60 mean cuts versus 1.74 for GNN-raw and 1.09 for SAT-min-k. This shows that on the same set of graphs, the GNN-guided approach improves the learned split, and not just feasibility, compared to pure GNN predictions.

Regarding runtime, GNN-raw is the overall fastest strategy, partitioning takes 0.4ms and SAT mapping 0.07s, matching greedy’s per-partition cost since both submit exactly one SAT check per partition. GNN-guided (partitioning 4 ms, SAT mapping 0.24s) is approximately 50 $\times$ faster end-to-end than SAT-min-k (12.1 s total), while the topological strategy (0.50 s SAT mapping) is

Table 2: Benchmark results. (a) full 150-graph evaluation set; (b) restricted to the 106 graphs solved by GNN-RAW, the like-for-like subset for the GNN-RAW vs. GNN-GUIDED comparison. † Greedy results are reported on its successes only (23 in (a), 20 in (b)) and are not directly comparable.

(a) Full 150-graph evaluation set.					
	SAT-MIN-K	Greedy	Topological	GNN-RAW	GNN-GUIDED
Success rate	100%	15.3%	92.7%	70.7%	98.0%
Mean edge cuts	1.31 ± 0.09	$1.52 \pm 0.37^\dagger$	3.22 ± 0.25	1.74 ± 0.17	2.15 ± 0.20
Partitioning (s)	0.581 ± 0.396	0.005 ± 0.001	0.007 ± 0.001	0.0004 ± 0.00001	0.004 ± 0.0004
SAT mapping (s)	11.508 ± 2.276	0.071 ± 0.004	0.498 ± 0.047	0.071 ± 0.002	0.243 ± 0.039
SAT checks / part.	80.59 ± 12.40	1.00 ± 0.00	3.71 ± 0.21	1.00 ± 0.00	2.36 ± 0.11

(b) 106-graph subset solved by GNN-RAW.					
	SAT-MIN-K	Greedy	Topological	GNN-RAW	GNN-GUIDED
Success rate	100%	18.9%	94.3%	100%	100%
Mean edge cuts	1.09 ± 0.06	$1.55 \pm 0.41^\dagger$	3.00 ± 0.29	1.74 ± 0.17	1.60 ± 0.17
Partitioning (s)	0.089 ± 0.013	0.005 ± 0.001	0.005 ± 0.0005	0.0003 ± 0.00001	0.002 ± 0.0001
SAT mapping (s)	7.923 ± 1.265	0.061 ± 0.004	0.393 ± 0.044	0.060 ± 0.002	0.133 ± 0.028
SAT checks / part.	70.44 ± 8.87	1.00 ± 0.00	3.60 ± 0.24	1.00 ± 0.00	2.01 ± 0.02

Strategies. SAT-MIN-K: exhaustive search over partition assignments; optimal reference. Greedy: single-pass list scheduling, no repair (fast baseline). Topological: topology-aware partitioner with all nodes seeded to one partition; feasibility repair only. GNN-RAW: raw GNN predictions submitted directly, without feasibility checks or repair. GNN-GUIDED: topology-aware partitioner seeded with GNN predictions (proposed method).

slower than GNN-GUIDED despite producing worse partitions, because a poor seed drives more feasibility-repair iterations and consequently more SAT checks per partition.

4 Conclusions

4.1 Results achieved and individual contribution

Overall, I have successfully implemented a complete pipeline for DFG-to-CGRA mapping guided by a learned model. The work progressed from state-of-the-art study to a complete experimental prototype combining data generation, exact labelling, learned prediction, repair and SAT-based spatial validation. The central result is that learned guidance improved the feasibility and cut quality of the practical partitioner while remaining much faster than exhaustive search.

The INForum 2026 submission (under review) is a project outcome, it communicates the research contribution and presents the results, whereas this report documents the complete internship process and my individual work. The Capstone Project was carried out individually in a team setting: I implemented

and evaluated the mapping pipeline and wrote this report. Francisco developed the separate downstream code-generation project, while the supervisors guided technical decisions, reviewed results and co-authored/reviewed the scientific paper.

4.2 Lessons learned

This internship has been a valuable learning experience, both technically and professionally. On the technical side, I gained experience in neural networks, graph neural networks, graph theory, accelerator architectures, and SAT solvers. The biggest takeaway for me was the importance of reading the state of the art. I learned how to search for papers, read them, and extract the relevant information to apply it to my own work. I also learned how to design and implement a complete pipeline for a complex problem, from data generation to model training and evaluation.

Working in a research team has also taught me valuable professional skills. I learned how to work autonomously on an open-ended research task, how to prepare concise weekly presentations, and how to receive and act on critical feedback.

4.3 Limitations and future work

This work has several limitations that should be addressed in future research. The current approach was evaluated on a homogeneous 4×4 CGRA and synthetic DFGs, which may not fully represent real-world scenarios. Future work could explore the evaluation of the proposed method on compiler-derived DFGs, larger and heterogeneous CGRAs, and explicit modelling of memory-transfer costs. Additionally, support for temporal or modulo scheduling could be investigated, as well as the prediction of cut probabilities per edge in addition to per-node partition indices.

4.4 Acknowledgements

This work is financed by National Funds through the FCT – Fundação para a Ciência e a Tecnologia, I.P. (Portuguese Foundation for Science and Technology) within the project AIMaCoV, with reference 2023.14899.PEX (<https://doi.org/10.54499/2023.14899.PEX>).

References

- [1] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.
- [2] Mihai Liviu Despa. Comparative study on software development methodologies. *Database Systems Journal*, 5(3):37–56, 2014.

- [3] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [4] GitLab Inc. GitLab. <https://gitlab.inescotec.pt/>, 2026. [Online; accessed 18 May 2026].
- [5] INForum. INForum 2026. <https://inforum.pt/>, 2026. [Online; accessed 26 June 2026].
- [6] Mingyang Kou, Jun Zeng, Xinyu Peng, Weiqing Ji, and Hailong Yao. Navimap: Partial order-guided neural architecture via deep q-networks for efficient cgra mapping. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, pages 364–371, 2025.
- [7] Jiangnan Li, Chang Cai, Yaya Zhao, Yazhou Yan, Wenbo Yin, and Lingli Wang. GRAFT: GNN-based adaptive framework for efficient CGRA mapping. In *2023 International Conference on Field Programmable Technology (ICFPT)*, pages 26–34, 2023.
- [8] Zhaoying Li, Dan Wu, Dhananjaya Wijerathne, and Tulika Mitra. LISA: Graph Neural Network based portable mapping on spatial accelerators. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 444–459. IEEE, 2022.
- [9] Bingfeng Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins. DRES-C: A retargetable compiler for coarse-grained reconfigurable architectures. In *2002 IEEE International Conference on Field-Programmable Technology, 2002. (FPT). Proceedings.*, pages 166–173, 2002.
- [10] Yukio Miyasaka, Masahiro Fujita, Alan Mishchenko, and John Wawrzynek. SAT-based mapping of data-flow graphs onto coarse-grained reconfigurable arrays. In *VLSI-SoC: Design Trends*, volume 621 of *IFIP Advances in Information and Communication Technology*, pages 113–131. Springer, 2021.
- [11] Omar Ragheb, Stephen Wicklund, Matthew Walker, Rami Beidas, Adham Ragab, Tianyi Yu, and Jason Anderson. Cgra-me 2.0: A research framework for next-generation cgra architectures and cad. In *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 642–649, 2024.
- [12] Slack Technologies. Slack. <https://slack.com/>, 2026. [Online; accessed 18 May 2026].
- [13] Cristian Tirelli, Lorenzo Ferretti, and Laura Pozzi. SAT-MapIt: A SAT-based modulo scheduling mapper for coarse grain reconfigurable architectures. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6. IEEE, 2023.